DHBW

Software Engineering
Program Design

Daniel Stimac

Program Design

The objective of this program design assignment is to develop a coherent software architecture and a detailed system design based on the System Requirements Specification (SRS) you created last quarter, together with the constraints defined for this task.

You are required to submit the following deliverables:

- A System Design Document (PDF) following the structure discussed in the lecture, describing the overall software architecture of the truck system. This document must contain all relevant architectural and design information and serves as the primary basis for assessment.
- The detailed design of the emergency stop system as part of the UML model below.
- An Enterprise Architect project file containing the UML diagrams you created for this program design. This file is submitted as supplementary material and will not be graded; all relevant content must be included in the System Design Document.

Schedule

- Due date for group selection:

  Send an e-mail to Stimac.Daniel@dozent.dhbw-ravensburg.de with all members of the group and a till **13.02.2026, 23:59**. Please define a group name and nominate one responsible member who will upload the final version on behalf of the group.

- Q & A Meeting:

  Q & A sessions is planned for February 16th and February 23rd

  Please send an advanced notice to Stimac.Daniel@dozent.dhbw-ravensburg.de if you have questions.

- **Due date for Submission: 02.03.2025, 23:59**

1. Program Design

- There will be a total of 100 points that can be achieved

Checkpoint	Max. Points achievable
Document the Architecture	75
Detailed Design	25

Architecture Documentation [75 Points]

Sub-Section	Expected Diagrams	Points
Overview	Context Diagram (Component or Package Diagram) modeling the overall truck system with system boundaries (e.g. connection to HW / PC application)	10
Subsystem decomposition: subsystem decomposition responsibilities of individual subsystems	Decompose the software architecture with a Package or Component Diagram into sub-system. Decompose each sub-system into a class-diagram. The class diagram shall contain attributes and public operations for the classes. Include here the discussion on potential architecture alternatives and the rational why your team has agreed on this specific architecture.	25
Hardware/software mapping: how subsystems are assigned to hardware	Provide a deployment diagram how the software sub-systems are mapped to hardware.	5
Global software control: initiation of requests subsystem synchronization concurrency	Create at least one sequence diagrams that model the interaction of the “Emergency Stop” with your subsystem, the sequence diagrams shall re-use the classes and interfaces you have created.	20
Boundary conditions: start-up, shutdown, and error behavior	Create at least one sequence diagram that models: start-up, shutdown and one error scenario.	15

Detailed Design

Select the class implementing most of the functionality “Emergency Stop” from your model and completely specify the detailed design. The detailed design needs to be specified in a detailed enough way so that all information that is required to implement the software is captured in the model.

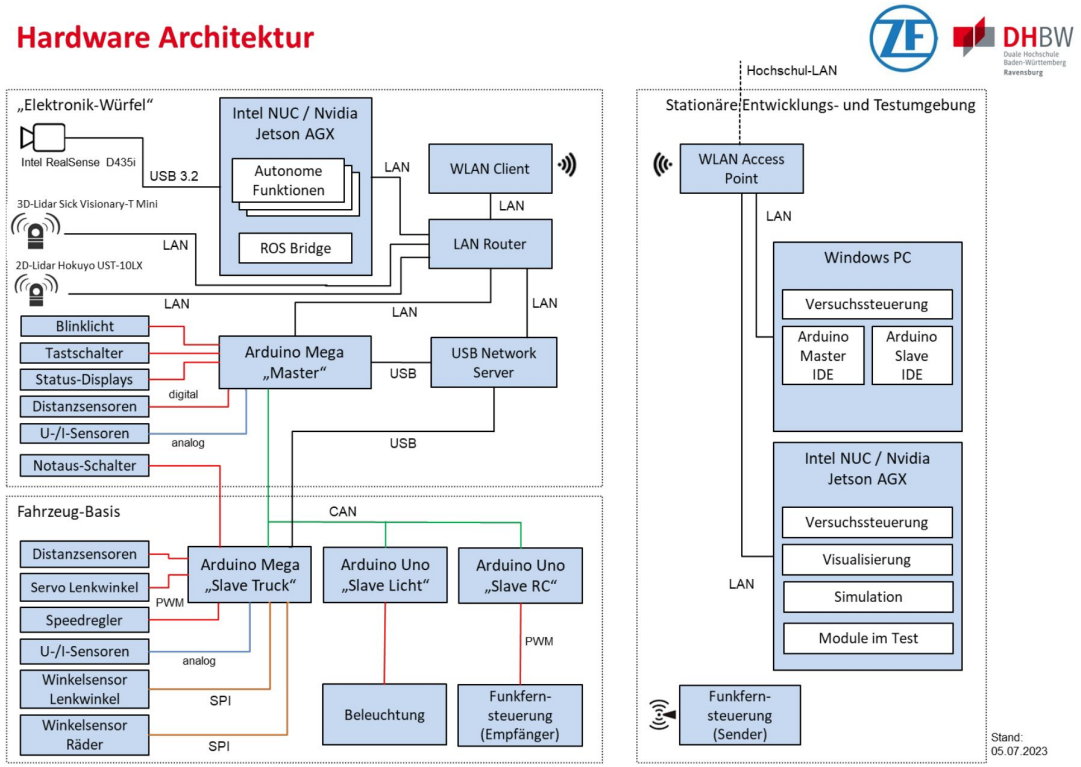
Create a Package “Detailed Design” that shall contain all diagrams created for this step.

The detail design shall include the following:

- The class shall be refined with private methods
- There shall be a sequence diagram specifying how the class is used (how the external interfaces are called) and what dependencies the class has, as well as how the internal (private) methods are called.
- For each method of the class there shall be either a state-machine diagram or an activity diagram created that models the behavior of the class. Use the note / comment feature of the model elements to specify additional information required for implementation. If your state-machine is implemented on the level of the class and controlled by several methods, only one state machine diagram is sufficient.

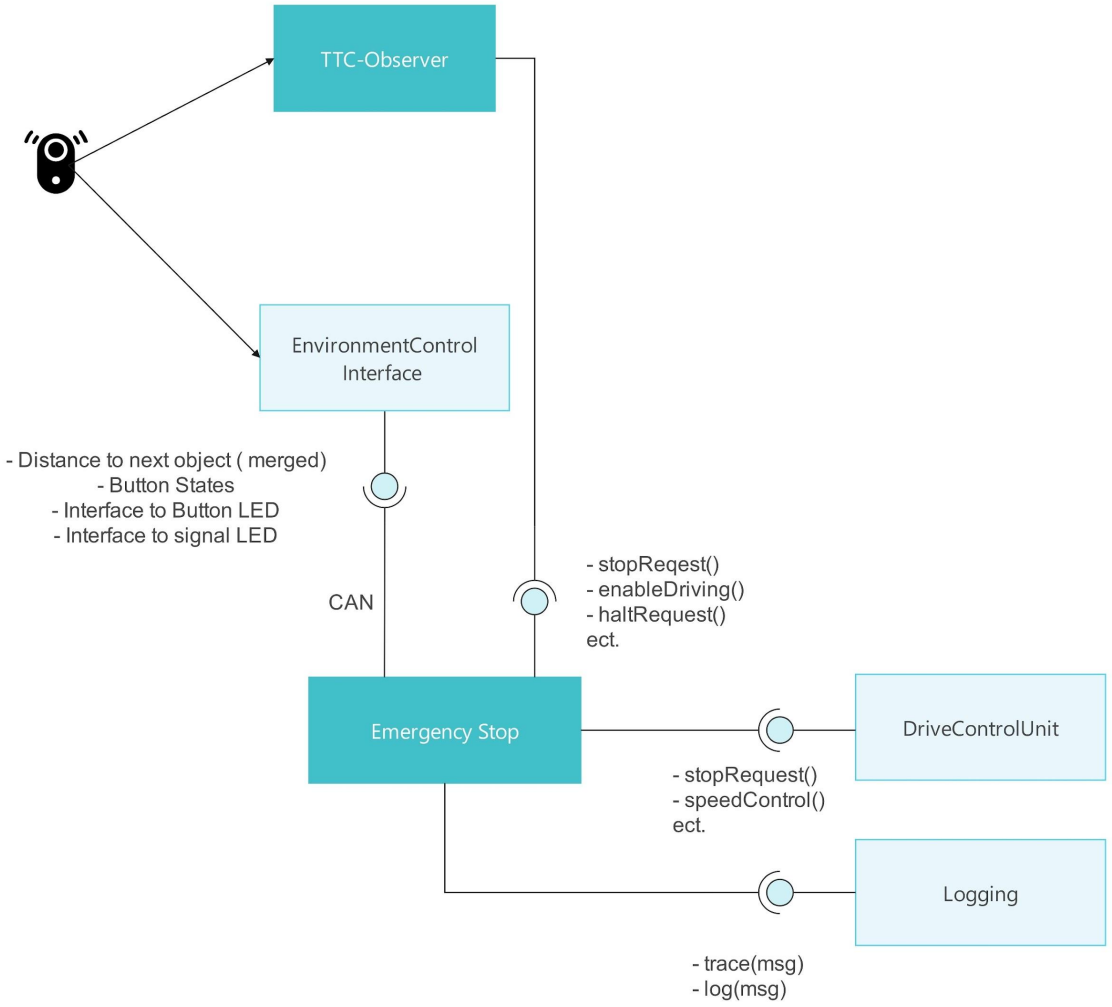
Our System

- Hardware Architekture
  - Jetson nano
    - AD fuctionality and sensor fusion
  - Arduino “Master”
    - Hardware access (LED, Button)
    - Provide basic distance and motion information
  - Arduino “Slave”
    - Drive Control
    - Emergency Stop



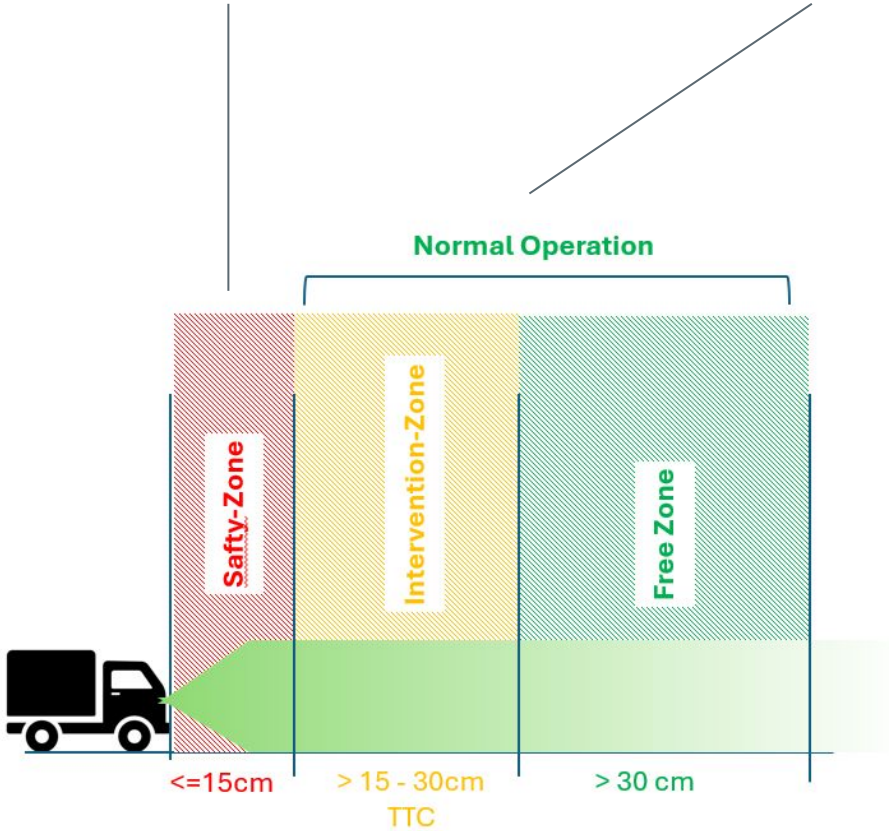
Component Overview

- Emergency Stop
  - safty monitoring
- TTC-Observer
  - Handle all AD operation modes. (TTC observation)

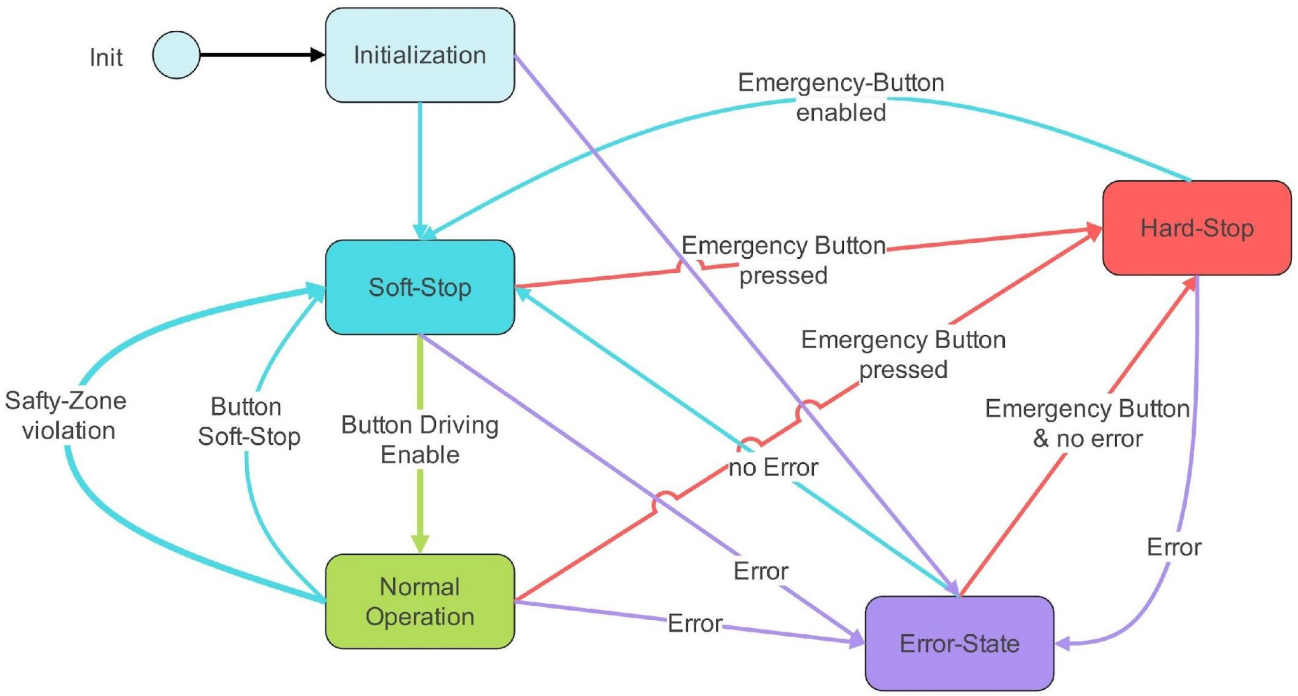


Operation Zones

- Safty-Zone
  - Emergency Stop, release via Button
- Normal Operation
  - Auto recover via TTC-Observer
- Emergency Stop via Soft-Stop Button and Emergency-Button (Pilz-Taster)



State-Diagram for Emergency Stop Component



Normal Operation:
- Handle alle Break and acceleration signals.
- TTC intervention is done by AD logic

Agenda:
Emergency Button = Pilz-Taster
Emergency Button -> Button pressed
Error -> Error detected

Basic requirements

- Emergency Button control (Pilz-Taster), Signal emergency stop via LED
- Soft-Stop Button Control and Button LED control
- Error Logging